ECOLE POLYTECHNIQUE DE BRUXELLES

INFO-H518: Immersive Multimedia Technologies I

Exercises INFO-H518: Part II

Niccolò Benedetto Mat. 626872
Paolo Bordis Alberto Mat. 628838

Prof. Lafruit Gauthier

Prof. Losfeld Armand

November, 2025

Contents

1 View Synthesis **2**

1.0.1 Q_V1 2

1.0.2 Q_V2 10

1.0.3 Q_V3 14

2 Iterated Closest Point **17**

2.0.1 Q_P1 17

2.0.2 Q_P2 19

2.0.3 Q_P3 19

3 COLMAP **24**

3.0.1 Q_C1 24

3.0.2 Q_C2 26

3.0.3 Q_C3 28

4 Zhang's Method **30**

4.0.1 Q_Z1 30

4.0.2 Q_Z2 31

4.0.3 Q_Z3 32

List of Figures

1.1 View left image 2
1.2 Disparity image 3
1.3 Mid-view forward 9
1.4 Mid-view backward 10
1.5 Mid-view backward t=0.00 11
1.6 Mid-view backward t=0.25 11
1.7 Mid-view backward t=0.50 12
1.8 Mid-view backward t=0.75 12
1.9 Mid-view backward t=1.00 13
1.10 Looking Glass moving right 13
1.11 Looking Glass moving left 14
1.12 PSNR outcome 16

2.1 Initial state: loading of the point clouds. 18
2.2 Result of RANSAC coarse registration. 18
2.3 Final result of the ICP with the color of cloud point. 19

3.1 COLMAP sfm on 10 images 25
3.2 COLMAP sfm on 20 images 25
3.3 COLMAP sfm on 38 images 26
3.4 COLMAP detph map 29

1 View Synthesis

1.0.1 Q_V1

Question: Provide you source code with forwarding and backward pixel referencing. Explain the difference in result between these two modes, while referring to the pieces of code where that difference appears (some might be based on the part of code that app)

In the following section we present our produced code in order to creates the view synthesis mini-project. We will discuss about the input image used, we present the logic of each function of the program and at the end we will analyze the outcomes.

As images input we used the following, extracted from the Middlebury dataset (link here).



Figure 1.1: View left image



Figure 1.2: Disparity image

Image and disparity loading A single reference image and its disparity are sufficient to synthesize a virtual view because disparity provides the horizontal reprojection of visible 3D points. However, regions that are occluded in the reference view cannot be reconstructed without the second image, which explains, as we will see, the presence of holes or artifacts in the synthesized mid-view.

The first function of the program is about loading the input images by using OpenCV.

```
1      def load_image_bgr(path: str) -> np.ndarray:
2      img = cv2.imread(path, cv2.IMREAD_COLOR)
3      if img is None:
4          raise FileNotFoundError(f"Cannot read image: {path}")
5      return img
```

Listing 1.1: Load images

The loading disparity function basically loads the disparity map provided by the Middlebury dataset. The disparity is read as is given to preserve its original bit depth. If needed, a multi-channel disparity image is converted to grayscale. The output is a `float32` array where values are expressed directly in pixels units, with zero indicating invalid or unknown disparity.

```
1
2  def load_disparity(path: str) -> np.ndarray:
3
```

```
    ext = os.path.splitext(path)[1].lower()
    if ext == ".npy":
        disp = np.load(path).astype(np.float32)
        return disp

    # Read as-is (keeps bit depth)
    disp_raw = cv2.imread(path, cv2.IMREAD_UNCHANGED)
    if disp_raw is None:
        raise FileNotFoundError(f"Cannot read disparity: {path}")

    # If 3-channel, convert to gray
    if disp_raw.ndim == 3:
        disp_raw = cv2.cvtColor(disp_raw, cv2.COLOR_BGR2GRAY)

    disp = disp_raw.astype(np.float32)

    # If it's 16-bit disparity from some tools, it might be scaled.
    # We don't know the scaling, so we provide a robust normalization
       option later.
    return disp
```

Listing 1.2: Load disparity

Forward mapping The next function is about implementing a simple hole-filling strategy used after forward warping. The idea is that pixels were not assigned during the mapping (i.e. holes) are filled by propagating the nearest valid pixel value along each image row.

```

def hole_fill_rowwise(img: np.ndarray, mask: np.ndarray) ->
    np.ndarray:
    out = img.copy()
    h, w = mask.shape

    for y in range(h):
        valid = mask[y].astype(bool)
        if valid.all():
```

```
            continue
        # indices of valid pixels
        xs = np.where(valid)[0]
        if xs.size == 0:
            continue

        # Fill each x by nearest valid x in the row
        # Efficient approach: forward pass then backward pass
        last = -1
        for x in range(w):
            if valid[x]:
                last = x
            else:
                if last != -1:
                    out[y, x] = out[y, last]

        last = -1
        for x in range(w - 1, -1, -1):
            if valid[x]:
                last = x
            else:
                if last != -1:
                    # choose closer between left-filled and
                       right-filled
                    # if left-fill didn't exist, it already remains
                       from right
                    out[y, x] = out[y, last]

    return out
```

Listing 1.3: Holes filling

Forward view synthesis This function implements forward pixel referencing. Basically each source pixel (x, y) is projected to a target position

$$x' = x - t \cdot \alpha \cdot d(x, y)$$

where $d(x, y)$ is the disparity in pixels, $t \in [0, 1]$ controls the virtual viewpoint position, and α scales the baseline. In addition this function deals with the occlusion problem: when multiple pixels map to the same target location we implement a z-buffer strategy or in other words we keep the pixel with the largest disparity (closest to the camera). At the end, holes caused by unassigned target pixels are filled using hole filling procedure.

```
1
2  def synthesize_forward(ref_bgr: np.ndarray, disp: np.ndarray, t:
       float, alpha: float) -> np.ndarray:
3
4      h, w = disp.shape
5      out = np.zeros_like(ref_bgr)
6      valid = np.zeros((h, w), dtype=np.uint8)
7
8      # z-buffer: store "closest" (largest disparity) that wrote each
           target pixel
9      zbuf = np.full((h, w), -1e9, dtype=np.float32)
10
11     # Precompute shift field
12     shift = t * alpha * disp  # float32
13
14     for y in range(h):
15         for x in range(w):
16             d = disp[y, x]
17             if d <= 0:
18                 continue
19
20             xp = int(round(x - shift[y, x]))
21             if 0 <= xp < w:
22                 # z-buffer test: closer = higher disparity
23                 if d > zbuf[y, xp]:
24                     zbuf[y, xp] = d
25                     out[y, xp] = ref_bgr[y, x]
```

```
                    valid[y, xp] = 1

    # Fill holes (unassigned pixels) with a simple row-wise strategy
    out_filled = hole_fill_rowwise(out, valid)
    return out_filled
```

Listing 1.4: Synthesize forward

Backward view synthesis This function implements backward (or inverse) pixel referencing:

- for each pixel (x', y), an approximate source coordinate is computed as

$$x = x' + t \cdot \alpha \cdot d(x', y)$$

- the reference image is sampled at this location using biliniear interpolation.

This approach ensures that every target pixel is assigned a value, thus avoiding holes. However, the inverse mapping is only approximate, since the disparity used corresponds to the target pixel rather than the true source surface.

```

def synthesize_backward(ref_bgr: np.ndarray, disp: np.ndarray, t:
    float, alpha: float) -> np.ndarray:

    h, w = disp.shape
    shift = t * alpha * disp

    # Target grid
    xs, ys = np.meshgrid(np.arange(w, dtype=np.float32), np.arange(h,
        dtype=np.float32))

    # Approximate inverse warp: sample from source coordinate
    src_x = xs + shift
    src_y = ys

    out = bilinear_sample_bgr(ref_bgr, src_x, src_y)
    return out
```

Listing 1.5: Synthesize backward

Backward mapping Samples color values from the reference image at floating-point coordinates using bilinear interpolation. This provides smoother results compared to nearest-neighbor sampling and is used during backward warping.

```
1
2  def bilinear_sample_bgr(img: np.ndarray, x: np.ndarray, y:
       np.ndarray) -> np.ndarray:
3
4      h, w = img.shape[:2]
5      x0 = np.floor(x).astype(np.int32)
6      y0 = np.floor(y).astype(np.int32)
7      x1 = x0 + 1
8      y1 = y0 + 1
9
10     x0 = np.clip(x0, 0, w - 1)
11     x1 = np.clip(x1, 0, w - 1)
12     y0 = np.clip(y0, 0, h - 1)
13     y1 = np.clip(y1, 0, h - 1)
14
15     wx = (x - x0).astype(np.float32)
16     wy = (y - y0).astype(np.float32)
17
18     Ia = img[y0, x0].astype(np.float32)
19     Ib = img[y0, x1].astype(np.float32)
20     Ic = img[y1, x0].astype(np.float32)
21     Id = img[y1, x1].astype(np.float32)
22
23     wa = (1 - wx) * (1 - wy)
24     wb = wx * (1 - wy)
25     wc = (1 - wx) * wy
26     wd = wx * wy
27
28     out = Ia * wa[..., None] + Ib * wb[..., None] + Ic * wc[...,
```

```
            None] + Id * wd[..., None]
29      return np.clip(out, 0, 255).astype(np.uint8)
```

Listing 1.6: Sampling BGR

The program main pipeline expects:

- loads the left image and the Middlebury disparity map;
- ensures valid image and disparity dimension;
- uses disparity values directly in pixel units;
- Synthesizes a virtual mid-view using both forward and backward mapping;
- saves the resulting images for visual inspection.

By running the program produced we obtain the following image output:



Figure 1.3: Mid-view forward



Figure 1.4: Mid-view backward

Let us analyze this results. The synthesized views clearly indicate the fundamental differences between forward and backward pixel referencing.

The backward-synthesized image is globally continuous and contains very few holes, since every target pixel is explicity sampled from the reference image. Neverthless there are visible horizontal distortion and artifacts near strong depth discontinuities, such as object boundaries between foreground and background pots. These flaws appears because the inverse correspondence is only approximated: the disparity used for sampling does not necessarily correspond to the correct visible surface, and bilinear interpolation blends pixels over depth edges.

Instead the forward synthesized image shows sharper object boundaries. In addition, thanks to the use of a z-buffer that favors closer surfaces, consistent occlusion are handled way better. Howevert this method produces numerous holes and cracks, especially in regions that become visible from the virtual point. We can say that these holes are an intrinsic limitation of forward referencing, as some target pixels are not reached by any source pixels. And the simple row-wise hole filling is not enought to eliminate these artifacts.

1.0.2 Q_V2

Question: Provide the output images and show the results on the Looking Glass display for visual inspection.

First we display the result obtained by using the Looking Glass online tool. Indeed we used Looking Gloves (link here) by handing a backward sequence with $t = 0.00$, $t = 0.25$, $t = 0.50$, $t = 0.75$ and $t = 1.00$ generated with the viewsynthesis previous code above.



Figure 1.5: Mid-view backward t=0.00



Figure 1.6: Mid-view backward t=0.25



Figure 1.7: Mid-view backward t=0.50



Figure 1.8: Mid-view backward t=0.75



Figure 1.9: Mid-view backward t=1.00

By doing that we obtain a quilt image named:

$$2025-12-30_15-17-17_qs8x1a0.75.jpg$$

Basically a quilt image is a horizontal sequence of different views where each of these shows a slightly shifted viewpoint. The filename tell us we deal with a quilt with 8 views in 1 row ($8x1$). Now this quil image represents the input for a Looking Glass display. Hence we uploaded it to Looking Glass Blocks (link here) in order see the how this browser interaction emulates what physical Looking Glass does optically. The result is the following one if we move the mouse towards the right:



Figure 1.10: Looking Glass moving right

while moving towards the left:



Figure 1.11: Looking Glass moving left

So let's try to analyze what's happening. We synthesized our views from a single reference image (the left view). When we scroll the mouse in Looking Glass Blocks we are just changing the virtual viewpoint left ↔ right. By scrolling the mouse to the right we are basically moving the viewpoint toward the original left camera. This is translated in fewer missing pixels and holes disappearance. Instead if we move the mouse in the opposite direction, we are just moving the virtual viewpoint away from the left camera. So as result we will have holes appear and with increasining viewpoint displacement most pixels unmapped (black image).

1.0.3 Q_V3

Question: Calculate the PSNR (or IV-PSNR, cf. attached reference [X]) for the mid-viewpoint image. Think of a good strategy to do this right; please explain it.

From the reference [X] we comprehend that IV-PSNR is a quality metric specifically designed for immersive video and synthesized views. Differently from the stadard PSNR, it compensates for small reprojection-induced pixel shifts by checking for the most similar pixel in a local neighborhood (tipically 5×5) and accounts for global color differences. Anyway IV-PSNR requires a reference view at the same viewpoint and therefore is not applicable to evaluating a synthesized mid-view in our case. Neverthless PSNR can be computed by synthesizing an existing input view and comparing it with the original.

Now we know that Middlebury 2006 dataset provides multiple real camera views (view 0 to view 6) which enables a proper evaluation. The mid-view between view1 and view5 corresponds to the real image view3. Therefore a virtual mid-view was synthesized from view1 using the provide disparity map and compared against the ground-truth view3 image.

We see how PSNR is computed between the synthesized and ground-truth images, both on the fukk RGB image and on the luminance (Y) component.

We added the followin functions to the previous code. The first one is about computing the standard Peak Signal-to-Noise Ratio (PSNR) between two color images in the RGB (BGR) domain. The PSNR value is computed using the classic formula:

$$PSNR = 10log_{10}(\frac{MAX^2}{MSE})$$

.

So higher PSNR indicates closer similarity btweeen the synthesized image and the reference.

```
1
2 def psnr_rgb(img_a_bgr: np.ndarray, img_b_bgr: np.ndarray, peak:
    float = 255.0) -> float:
3     a = img_a_bgr.astype(np.float32)
4     b = img_b_bgr.astype(np.float32)
5     mse = np.mean((a - b) ** 2)
6     if mse == 0:
7         return float("inf")
8     return 10.0 * np.log10((peak ** 2) / mse)
```

Listing 1.7: PSNR function

The next function converts a BGR color image to its luminance (Y) component using the ITU-R BT.601 standard, which approximates human visual sensitivity. We extract the green, blue and red channels from the input image and converts to `float32`. The luminance value is computed as weighted sum of the RGB components:

$$Y = 0.299R + 0.587G + 0.114B$$

.

This function is important since distortions in luminance are generally more perceptible than chromatic distortions, so evaluating PSNR on the Y channel is often more meaningful perceptually.

```
1
2 def bgr_to_luma_y_bt601(img_bgr: np.ndarray) -> np.ndarray:
3     # ITU-R BT.601 luma (approx), using RGB coefficients; convert
        BGR->RGB first
```

```
    b = img_bgr[..., 0].astype(np.float32)
    g = img_bgr[..., 1].astype(np.float32)
    r = img_bgr[..., 2].astype(np.float32)
    # Y = 0.299 R + 0.587 G + 0.114 B
    return 0.299 * r + 0.587 * g + 0.114 * b
```

Listing 1.8: BGR to luma

The third function basically computes PSNR only on the luminance component of the images, rather than on all color channels.

```

def psnr_luma(img_a_bgr: np.ndarray, img_b_bgr: np.ndarray, peak:
    float = 255.0) -> float:
    ya = bgr_to_luma_y_bt601(img_a_bgr)
    yb = bgr_to_luma_y_bt601(img_b_bgr)
    mse = np.mean((ya - yb) ** 2)
    if mse == 0:
        return float("inf")
    return 10.0 * np.log10((peak ** 2) / mse)
```

Listing 1.9: PSNR luma

By running the program in order to look at the PSNR outcome we deal with the following output:

```
(lkg) PS C:\Users\nicco\Desktop\ULB\courses\first term\INFO-H518 Immersive Multimedia Technologies I\EX_INFO-H518_Part2\viewsynthesis> & C:\Users\nicco\anaconda3
\envs\lkg\python.exe "c:/Users/nicco/Desktop/ULB/courses/first term/INFO-H518 Immersive Multimedia Technologies I/EX_INFO-H518_Part2/viewsynthesis/viewsys.py"
disp dtype: float32 min: 0.0 max: 181.0
Saved:
  c:\Users\nicco\Desktop\ULB\courses\first term\INFO-H518 Immersive Multimedia Technologies I\EX_INFO-H518_Part2\viewsynthesis\vase_forward.png
  c:\Users\nicco\Desktop\ULB\courses\first term\INFO-H518 Immersive Multimedia Technologies I\EX_INFO-H518_Part2\viewsynthesis\vase_backward.png

-----PSNR-----
Forward RGB PSNR:  18.682161
Backward RGB PSNR:  17.263763
Forward Luma PSNR:  18.564919
Backward Luma PSNR:  17.18058
```

Figure 1.12: PSNR outcome

We observe that the PSNR values are 18.68 dB (forward) and 17.26 dB (backward) on RGB, and 18.56 dB (forward) and 17.18 dB (backward) on luma. Forward mapping archieves higher PSNR because it preserves sharper object boundaries and occlusion. Instead backward mapping introduces smoothing artifacts near depth discontinuities due to interpolation. Anyway, althuogh backward mapping produces visually smoother images with fewer holes, the PSNR standard penalizes such geometric distortions. This naturally leads to think of IV-PSNR as a better metric.

2 Iterated Closest Point

2.0.1 Q_P1

Question: Explain the various steps you take (you may first use content provided by the teacher before taking the pictures by yourself).

To successfully merge the two partial views of the face, I followed a pipeline consisting of four main steps:

1. **3D Point Cloud Generation (Reprojection):** First, I converted the 2D data into 3D geometry. For both the Left and Right datasets, I used the provided RGB images and Disparity Maps. By applying the reprojection formula $Z = \frac{f \cdot B}{d}$ (using the camera intrinsic parameters), I generated two separate raw point clouds representing the left and right sides of the face.

2. **Preprocessing and Filtering:** Raw point clouds often contain noise and background elements (like the wall behind the subject) that can confuse the alignment algorithms. I applied a **depth-based filter** (keeping only points within a specific Z-range from the camera, e.g., the nose area) and a **Statistical Outlier Removal** filter to clean the data and isolate the facial geometry from the background.

3. **Coarse Alignment (Global Registration via RANSAC):** Since the two point clouds start in different coordinate systems, they do not overlap initially. To bring them close together, I performed a Global Registration step. I computed **FPFH features** (Fast Point Feature Histograms) for both clouds to describe the local geometry. Then, I used **RANSAC** (Random Sample Consensus) to find an initial transformation matrix that roughly aligns the matching features (e.g., matching the nose on the left cloud with the nose on the right).

4. **Fine Alignment (Iterative Closest Point - ICP):** Finally, starting from the rough alignment provided by RANSAC, I applied the **ICP algorithm**. This step iteratively minimizes the distance between corresponding points (or point-to-plane distance) in the overlapping regions. This refined the transformation matrix to achieve a seamless stitch between the left and right halves of the face.

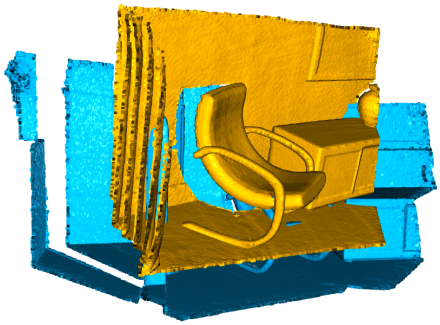


Figure 2.1: Initial state: loading of the point clouds.

Figure 2.1 shows the initial loading of the two point clouds. The source cloud has been manually rotated and translated to simulate a misalignment relative to the target.

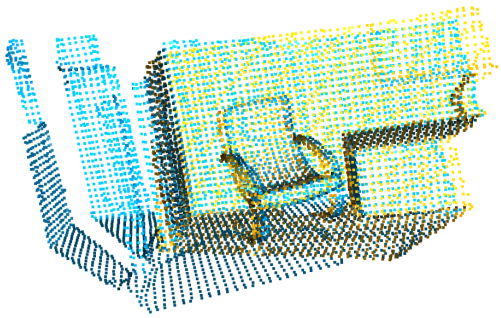


Figure 2.2: Result of RANSAC coarse registration.

As shown in Figure 2.3, the RANSAC algorithm successfully performed a coarse alignment. The source point cloud is now roughly aligned with the target cloud.

However, a slight misalignment is still visible (creating artifacts such as double borders or "ghosting" effects on the object boundaries). To solve this, the Iterative Closest Point (ICP) algorithm is applied. ICP minimizes the distance between corresponding points (or point-to-plane distance) to refine the registration and perfectly merge the two point clouds into a single surface.

2.0.2 Q_P2

Question: Take a snapshot of your result from a middle viewpoint, e.g. using MeshLab.



Figure 2.3: Final result of the ICP with the color of cloud point.

2.0.3 Q_P3

Question: Provide your source code (possibly an extension of existing internet code), as well as the PLY or OBJ files of the input and output of your code. Explain the main processing steps succinctly (some might be based on code from the IPOL papers of Part I).

```
1
2 def convert_pfm_to_pc(disp_data, color_img, K, baseline):
3     h, w = disp_data.shape[:2]
4     fx, fy = K[0, 0], K[1, 1]
```

```
    cx, cy = K[0, 2], K[1, 2]

    mask_inf = np.isinf(disp_data)
    mask_valid = (disp_data > 0) & (~mask_inf)

    d_valid = disp_data[mask_valid]
    c_valid = color_img[mask_valid].astype(np.float32) / 255.0

    z = (fx * baseline) / d_valid
    cols, rows = np.meshgrid(np.arange(w), np.arange(h))
    u_valid = cols[mask_valid]
    v_valid = rows[mask_valid]

    x = (u_valid - cx) * z / fx
    y = (v_valid - cy) * z / fy

    points = np.column_stack((x, y, z))

    pcd = o3d.geometry.PointCloud()
    pcd.points = o3d.utility.Vector3dVector(points)
    pcd.colors = o3d.utility.Vector3dVector(c_valid)
    pcd.transform([[1, 0, 0, 0], [0, -1, 0, 0], [0, 0, -1, 0], [0, 0,
       0, 1]])

    if len(pcd.points) > 0:
        pcd, _ = pcd.remove_statistical_outlier(nb_neighbors=50,
           std_ratio=1.0)
    return pcd
```

Listing 2.1: Function to convert PFM files to Point Cloud objects

This is the primary script used to generate point clouds from the provided depth maps. For this experiment, I utilized a dataset containing **PFM (Portable FloatMap)** files. Note that while 'PFM' can refer to 'Printer Font Metrics', in this context, it strictly denotes the scientific floating-point image format used for high-precision depth data.

As previously mentioned, this script converts 2D pixel coordinates (u, v) and depth values into

3D points (x, y, z).

```
print("-" * 50)
print(":: RANSAC (Alignment Recovery)...")
voxel_size = 5.0

source_down = source.voxel_down_sample(voxel_size)
target_down = target.voxel_down_sample(voxel_size)
radius_normal = voxel_size * 2.0
radius_feature = voxel_size * 5.0

# Estimate normals for feature computation
source_down.estimate_normals(
    o3d.geometry.KDTreeSearchParamHybrid(radius=radius_normal,
        max_nn=30))
target_down.estimate_normals(
    o3d.geometry.KDTreeSearchParamHybrid(radius=radius_normal,
        max_nn=30))

# Compute FPFH features
source_fpfh = o3d.pipelines.registration.compute_fpfh_feature(
    source_down,
    o3d.geometry.KDTreeSearchParamHybrid(radius=radius_feature,
        max_nn=100))
target_fpfh = o3d.pipelines.registration.compute_fpfh_feature(
    target_down,
    o3d.geometry.KDTreeSearchParamHybrid(radius=radius_feature,
        max_nn=100))

# Execute RANSAC
ransac =
   o3d.pipelines.registration.registration_ransac_based_on_feature_matching(
    source_down, target_down, source_fpfh, target_fpfh, True,
    voxel_size * 3.0,
    o3d.pipelines.registration.TransformationEstimationPointToPoint(False),
    3, [
```

```
        o3d.pipelines.registration.CorrespondenceCheckerBasedOnEdgeLength(0.9),
        o3d.pipelines.registration.CorrespondenceCheckerBasedOnDistance(
            voxel_size * 3.0)
    ], o3d.pipelines.registration.RANSACConvergenceCriteria(4000000,
       0.999))

print(f"   > RANSAC Fitness: {ransac.fitness:.4f}")
source.transform(ransac.transformation)
visualize_result(source, target, "2. Post RANSAC",
   use_real_colors=False)

print("-" * 50)
print(":: ICP (Fine Alignment)...")
source.estimate_normals(
    o3d.geometry.KDTreeSearchParamHybrid(radius=radius_normal,
       max_nn=30))
target.estimate_normals(
    o3d.geometry.KDTreeSearchParamHybrid(radius=radius_normal,
       max_nn=30))

# Execute ICP Point-to-Plane
icp_fine = o3d.pipelines.registration.registration_icp(
    source, target, voxel_size * 0.4, np.identity(4),
    o3d.pipelines.registration.TransformationEstimationPointToPlane(),
    o3d.pipelines.registration.ICPConvergenceCriteria(
        relative_fitness=1e-6, relative_rmse=1e-6, max_iteration=50)
)

print(f"   > ICP Fitness: {icp_fine.fitness:.4f} | "
      f"RMSE: {icp_fine.inlier_rmse:.4f}")
source.transform(icp_fine.transformation)

# Merge and Save
merged = source + target
merged = merged.voxel_down_sample(voxel_size=1.0)

```

```
62 visualize_result(merged, o3d.geometry.PointCloud(),
63                  "3. FINAL RESULT (Real Colors)",
                        use_real_colors=True)
64 o3d.io.write_point_cloud("merged_face_perturbed.ply", merged)
```

Listing 2.2: RANSAC and ICP execution

The script above handles the execution of the RANSAC (Global Registration) and ICP (Fine Alignment) algorithms.

3 COLMAP

3.0.1 Q_C1

Question: Report (provide a trend on) how the qualityof the reconstruction changes with the number of photos used in the processing. How many input pictures (order of magnitude) are needed to reconstruct your 3D scene decently?

We used COLMAP's incremental structure-from-motion (sfm) on the courtyard dslr undistorted scene (dataset link here), repeating the pipeline with different numbers of input photos. And for each run we consider the following metrics:

- number of registered images (in other words how many camera poses COLMAP can estimate);
- number of 3D points;
- visual completeness of the scene.

By running the experiment using the first 10 image of the courtyard dataset COLMAP sfm produce exactly 10/10 registered images and 6249 3D points. By observing the result below we can see that reconstruction is stable and works but the point cloud is relatively sparse and the scene is limited because there are fewer viewpoints.

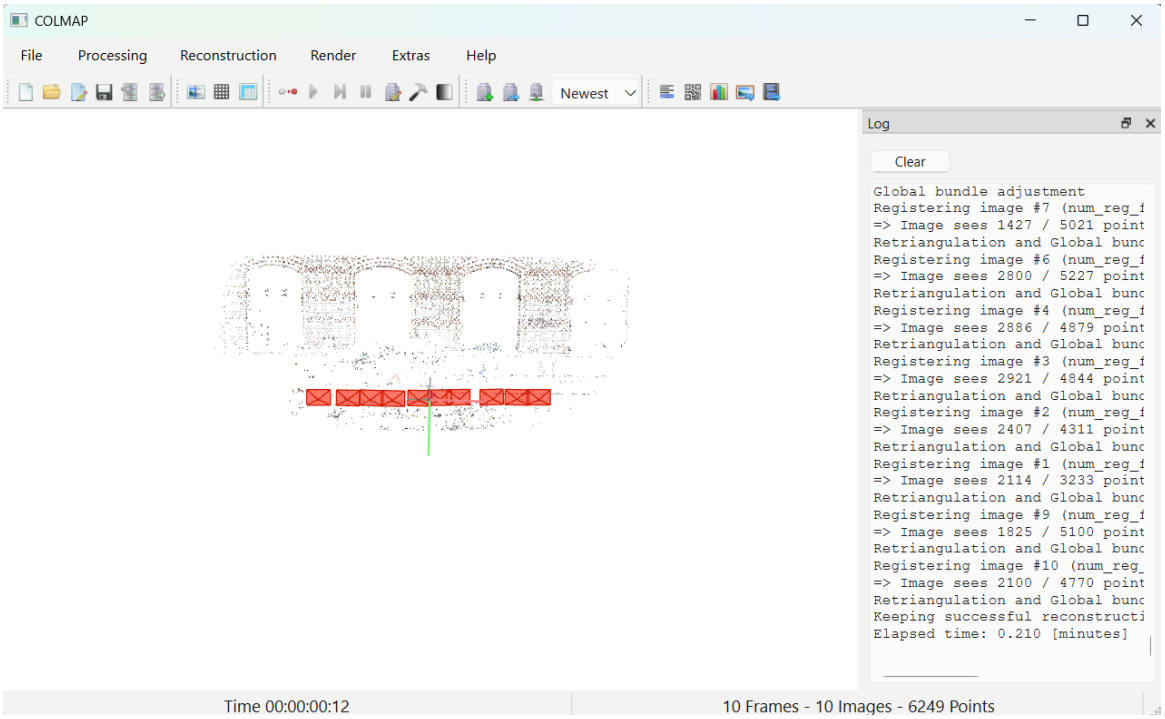


Figure 3.1: COLMAP sfm on 10 images

The 20 images experiment provides 20/20 registered images and 11297 3d points. We notice such clear improvement over the 10-image case due to more viewpoints that mean more triangulated points and a better scene coverage.

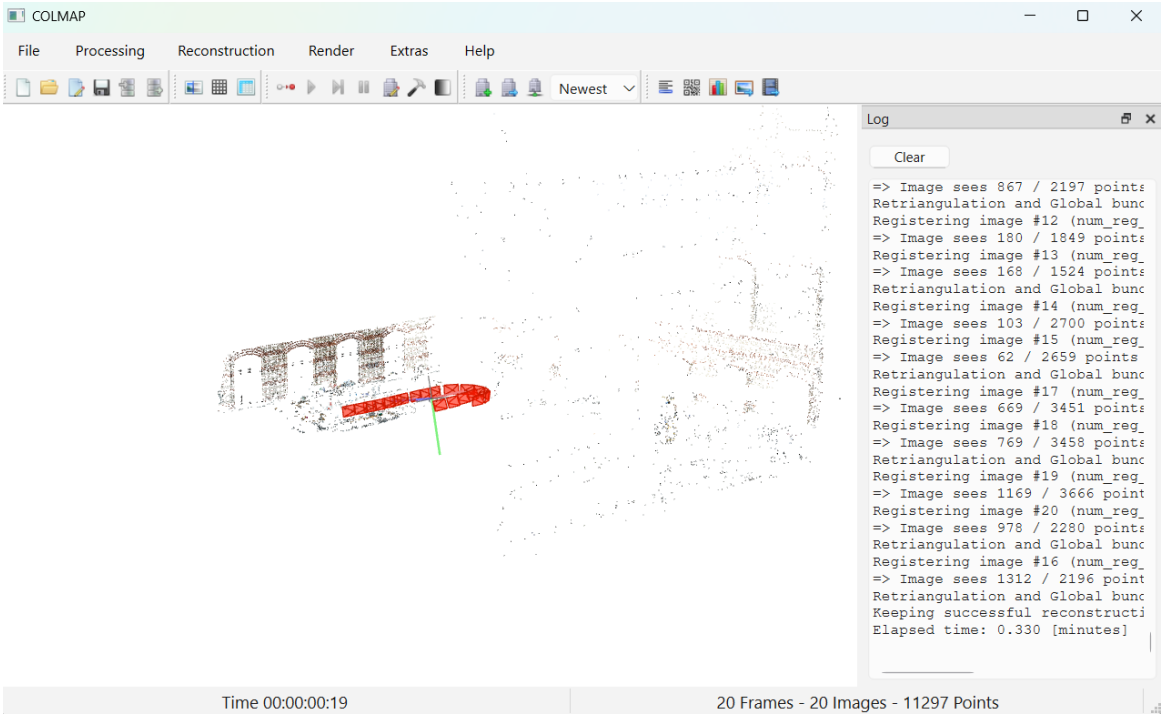


Figure 3.2: COLMAP sfm on 20 images

In the last experiment we used the all courtyard dataset (38 images). We obtain 38/38 registered

images and 23964 3D points. As expected this represent the best result: we observe such more complete geometry and the set of camera poses covers the scene better.

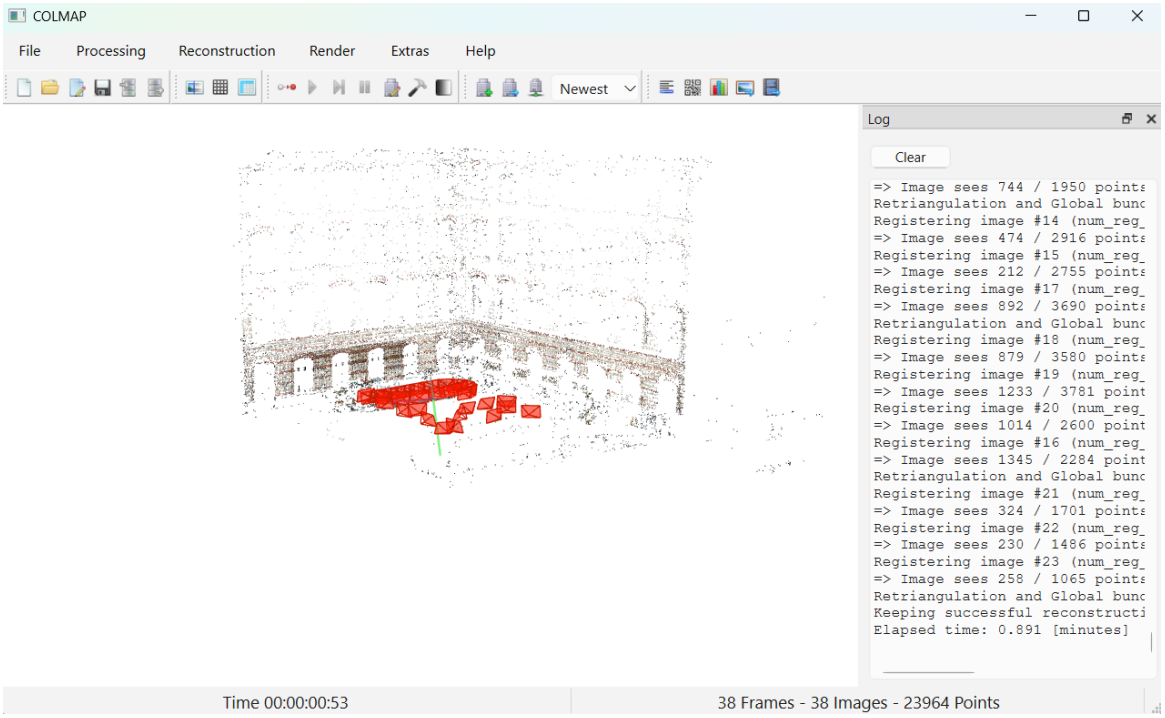


Figure 3.3: COLMAP sfm on 38 images

As trend we analyze how the quality changes with number of photos. Basically we have that:

- registration becomes more robust with more images;
- sparse point cloud density increases strongly with more images.

At the end of the day we able to say that, based on our result, at least a few tens of images (10^1) are needed to reconstruct the 3D scene decently. And around 30/40 images is a good threshold for a “decent” recontruction of this courtyard scene.

3.0.2 Q_C2

Question: Report intrinsics K and extrinsics (R|t) between the first and N-th input view, with N in the order of ¼ of the total number of input views you used for the 3D reconstruction. Also provide radial distortion parameters. Do all these values look reasonable to you?

The courtyard scene was reconstructed using COLMAP Incremental Structure-from-Motion with a total of $M = 38$ input images. Following the requirement to select N on the order of $\frac{1}{4}M$, we choose

$N = 10$. Intrinsic and extrinsic parameters were extracted by exporting the sparse reconstruction model to `txt` format and reading the values from `cameras.txt` and `images.txt`.

Camera Intrinsics The reconstruction uses a single camera model of type `SIMPLE_RADIAL` with image resolution $W \times H = 6205 \times 4135$. By reading the first line of `cameras.txt` we have:

$$1\ SIMPLE_RADIAL\ 6205\ 4135\ 3404.2480462112403\ 3102.5\ 2067.5\ -0.00076976581538170644$$

.

The intrinsic calibration matrix is

$$K = \begin{bmatrix} f & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 3404.248046 & 0 & 3102.5 \\ 0 & 3404.248046 & 2067.5 \\ 0 & 0 & 1 \end{bmatrix}.$$

The radial distortion model estimates a single coefficient

$$k_1 = -7.6977 \times 10^{-4}.$$

Camera Extrinsics COLMAP provides camera poses in the form of world-to-camera transformations,

$$x_{\text{cam}} = RX_{\text{world}} + t.$$

For the first view (image `DSC_0286.JPG`), the extrinsic parameters are:

$$R_1 = \begin{bmatrix} 0.987095 & 0.012289 & -0.159665 \\ -0.006207 & 0.999238 & 0.038535 \\ 0.160017 & -0.037046 & 0.986419 \end{bmatrix}, \quad t_1 = \begin{bmatrix} 2.038154 \\ 0.307223 \\ -2.191157 \end{bmatrix}.$$

For the N-th view (image `DSC_0295.JPG`), the extrinsic parameters are:

$$R_{10} = \begin{bmatrix} 0.972127 & 0.003380 & -0.234430 \\ 0.006523 & 0.999119 & 0.041456 \\ 0.234364 & -0.041830 & 0.971249 \end{bmatrix}, \quad t_{10} = \begin{bmatrix} -1.995184 \\ 0.280328 \\ -2.360570 \end{bmatrix}.$$

Relative Extrinsics Between Views To describe the rigid transformation between the first and the N-th camera frames, we compute the relative pose from camera 1 to camera 10 as

$$R_{10,1} = R_{10}R_1^\top, \qquad t_{10,1} = t_{10} - R_{10,1}\,t_1.$$

This generates:

$$R_{10,1} = \begin{bmatrix} 0.997053 & -0.011690 & -0.075814 \\ 0.012098 & 0.999915 & 0.004924 \\ 0.075750 & -0.005826 & 0.997110 \end{bmatrix}, \quad t_{10,1} = \begin{bmatrix} -4.189863 \\ -0.040739 \\ -0.328346 \end{bmatrix}.$$

We can claim that the estimated intrinsic parameters are reasonable: the principal point lies exactly at the image center, and the focal length magnitude is plausible for the given image resolution. The radial distortion coefficient is very close to zero, which is aligned with the use of undistorted input images. The relative rotation matrix is orthonormal and represents a small but meaningful viewpoint change.

3.0.3 Q_C3

Question: Extract and show the depth map for one view.

To extract a depth map, we used COLMAP's dense multi-view stereo pipeline. Since dense stereo reconstruction requires CUDA support, the computation was performed in a CUDA-enabled Linux environment using Google Colab. So the sparse SfM reconstruction and input images were transferred to Colab.

First, the `image_undistorter` was applied to prepare the dense workspace. Then, patch match stereo was run with geometric consistency enabled to compute per-view depth maps. The depth maps were generated in the directory `dense/stereo/depth_maps/`. We chose one depth map corresponding to the view `DSC_0286.JPG`, we extracted it and converted it to a 2D image for visualization.

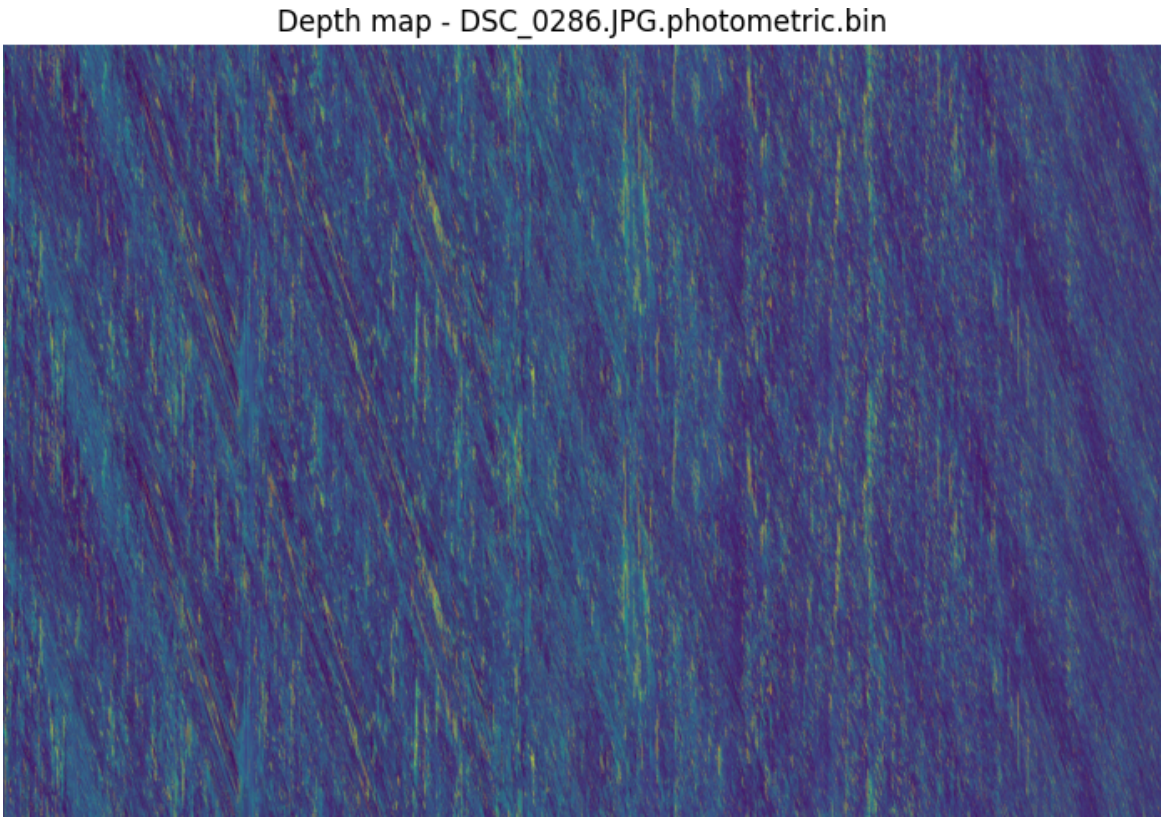


Figure 3.4: COLMAP detph map

Figure 3.4 shows the resulting photometric depth map. We observe that brighter colors correspond to regions closer to the camera, while darker colors indicate larger depth values. However we notice some noisy appearances as well due to repeated textures and varying illumination. So, overall, the depth map contains noise but still is asbel to capture meaningful geometric structure.

4 Zhang's Method

4.0.1 Q_Z1

Question: Which initial guess is given to the SfM in [Y,Z]? Explain how it is obtained.

We can claim that the initial guess consists of camera intrinsic matrix K and camera extrinsic parameters (R_i, t_i) for each image. These basically are the paramaters that will be later be refined by nonlinear optimization. The initial guess is obtained through planar homography estimation. Indeed, because the calibration is planar, all 3D points lie on a plane and we know that 2D image points correspond to 2D plane coordinates. This correspondence defines the homography H_i. So for each image i Zhang computes this:

$$H_i : plane \rightarrow image$$

Such step uses linear estimation.

The Zhang's key observation here is that each homography can be decomposed as:

$$H = K[r_1\ r_2\ t]$$

.

And since r_1 and r_2 are orthonormal vectors then two independent contraints per image are generated. So for multiple images, Zhang builds a linear system whose solution gives us an initial estimate of K.

At this point each homography H_i is decomposed and rotation and translation are fetched analytically.

These estimates are then used to initialize a nonlinear optimization that minimizes the reprojection error.

4.0.2 Q_Z2

Question: Find the correspondences between the theory syllabus text and the [Y,Z] references to explain the main processing steps. Your description can refer to page numbers and equations in the syllabus (consider this question rather as preparation for similar questions at the oral exam; do not give too many details in the report).

Zhang's calibration method directly follows the theoretical framework presented in the course syllabus, combining concepts from camera modeling, planar homographies, and structure from motion.

First, the camera model used in Zhang's method corresponds to the pinhole camera model introduced in the syllabus (Chapter II). The projection equation

$$x \sim K[R \mid t]X$$

assumes a constant intrinsic matrix K for all images, while the extrinsic parameters (R, t) vary for each view of the calibration pattern. This matches the standard camera model discussed in the theory lectures.

Second, the use of a planar calibration object allows the mapping between the object plane and the image to be described by a homography, as presented in the syllabus chapter on homographies (Chapter III). For each image, a homography H is estimated from point correspondences using linear techniques such as the Direct Linear Transformation. This homography can be expressed as

$$H = K[r_1 \; r_2 \; t],$$

where r_1 and r_2 are the first two columns of the rotation matrix.

Zhang exploits the orthonormality constraints on r_1 and r_2 to derive linear constraints on the intrinsic parameters. Indeed we will have:

$$r_1^T r_2 = 0, \quad \|r_1\| = \|r_2\|$$

and by considering these constraints from multiple homographies, we can obtain a linear system and solve it using least squares.

Now, once an initial estimate of the intrinsic matrix and the extrinsic parameters is obtained, Zhang applies a nonlinear optimization that minimizes the reprojection error over all images. This

final refinement step corresponds to the bundle adjustment process discussed in the Structure from Motion chapter (Chapter VII) of the syllabus.

At the end Zhang's method basically can be seen as an application of thecconcepts presented in the course, combining homography estimation, camera modeling, and nonlinear optimization into a complete calibration pipeline.

4.0.3 Q_Z3

Question: Is radial distortion estimation (and removal) embedded as with SfM from the theory lessons? Explain the similarities and/or differences.

We can claim that radial distortion is embedded in Zhang's calibration estimation in the same conceptually way to the treatment of distortion in Structure from Motion (SfM).

In both Zhang's method and SfM, radial distortion is explicitly modeled using distortion coefficients and is estimated through a nonlinear optimization that minimizes the reprojection error. In this sense, distortion estimation is fully integrated into the camera model.

However the main difference is placed in the initialization strategy. In Zhang's method, the intrinsic and extrinsic parameters are first estimated using planar homographies. Radial distortion parameters are introduced only in the final nonlinear step. Instead, SfM typically estimates camera intrinsics, extrinsics, distortion parameters, and 3D structure simultaneously, without a linear initialization.

Furthermore, Zhang's method benefits from a known planar calibration object, which reduces the number of unknowns and stabilizes the optimization. SfM, on the other hand, operates on unknown scene geometry and therefore requires a more general and computationally demanding optimization. As a result, Zhang's method can be seen as a constrained and specialized form of SfM, where radial distortion is estimated using the same principles but under powerful geometric assumptions.